

Documentación

Organización de computadores.

Memory.hdl

Para hacer este chip se tuvo primero que tener claro la teoría de la memoria principal del Hack. Esta teoría nos dice que memory Actúa como una interfaz unificada para acceder a RAM16K, Screen y Keyboard mediante una dirección global de 15 bits, Por tanto existen $2^{15} = 32768$ direcciones, desde 0 hasta 32767. Gracias a esta información podemos saber el siguiente mapa de memoria:

RANGO DE BITS	UNIDAD
0 – 16383	RAM16K
16384 – 24575	Screen
24576	Keyboard

- RAM16K contiene 16384 (2^{14}) posiciones, por lo que necesita 14 bits de dirección.
- Screen contiene 8192 (2^{13}) posiciones, por lo que necesita 13 bits de dirección.
- Keyboard se trata como un único registro de lectura.

Sabiendo que Memory recibe:

- in[16]: dato que se desea escribir.
- load: indica si se debe escribir.
- address[15]: dirección global.

Memory entrega:

- out[16]: dato leído.

Podemos inferir que la combinación de los últimos 2 bits de address (address[13] y address[14]) representará los siguientes intervalos de memoria:

Address[14]	Address[13]	INTERVALO DE MEMORIA
0	0	0 a 8191
0	1	8192 a 16383
1	0	16384 a 24575
1	1	24576 a 32767

Para poder procesar estos datos de entrada y saber cómo memory decidirá si accede a RAM16K, Screen o Keyboard se necesita saber la teoría del Mux y DMux.

Por la teoría anteriormente explicada sabemos que Memory sabrá acceder a que unidad mediante los últimos 2 bits de address. Con la siguiente tabla podremos evidenciar las combinaciones posibles de esos 2 bits y que combinación representa que unidad:

Address[14]	Address[13]	Unidad
0	0	RAM16K
0	1	RAM16K
1	0	Screen
1	1	Keyboard

Con esta tabla podemos evidenciar que con address[14] podemos filtrar si Memory accede a RAM16K o no. Esto nos sirve para poder hacer el primer DMux:

```
DMux(in=load , sel=address[14] , a=a1 , b=loadrest );
```

La variable a1 representará el load de RAM16K y loadrest servirá para el otro DMux.

Después de hacer ese primer DMux podemos evidenciar que el address[13] nos ayuda a distinguir si Memory accede a Screen si es 0 o a Keyboard si es 1. Esto significa que con la información anterior podemos inferir el siguiente DMux:

```
DMux(in=loadrest , sel=address[13] , a=b1 , b=c1 );
```

Representando las variables b1 el load de Screen

```
Screen(in=in , load=b1 , address=address[0..12],out=o1 );
RAM16K(in=in , load=a1 , address[0]=address[0..13], out=o2 );
```

Aquí es más que trivial entender la llamada de cada una de las unidades. Lo único que vale la pena recalcar la cantidad de bits de address[0...i] que se le ingresa a cada una de las unidades. A Screen solo se le ingresan 13 bits debido a que este contiene 2^{13} posiciones en memoria. A RAM16K se le ingresan 14 bits debido a que este contiene 2^{14} posiciones en memoria.

Podemos evidenciar que el Keyboard no tiene ningún dato de entrada:

```
Keyboard(out=o3 );
```

Esto es debido a que keyboard no necesita ninguna dirección interna debido a que se utiliza principalmente para lectura. Lo que implica que siempre dará una salida, solo que nosotros la “dejaremos pasar” con un Mux.

Luego de tener todas estas unidades definidas , Memory solo tiene una salida, así que debe de saber cual dejar pasar. Esto lo haremos con un Mux16. Para el Mux16 usaremos la misma teoría que los dos DMux del inicio:

Address[14]	Address[13]	ENTRADA	UNIDAD
0	0	A	RAM16K
0	1	B	RAM16K
1	0	C	Screen
1	1	D	Keyboard

Así con los 2 últimos bits de address sabremos Memory a que unidad debe de ingresar y que salida debe de dar.

CPU.hdl y Computer.hdl

CPU.hdl

Para hacer este chip se tuvo primero que tener clara la teoría de funcionamiento de la CPU Hack. A diferencia de Memory, donde el problema principal era decidir a qué unidad de memoria acceder, en CPU el problema consiste en interpretar una instrucción de 16 bits, ejecutar la operación indicada y decidir qué registros deben almacenar el resultado y cuál será la siguiente instrucción que debe ejecutar el computador.

La CPU recibe tres entradas principales: el dato proveniente de memoria, la instrucción actual y la señal de reset. A partir de estas entradas produce el dato que eventualmente será escrito en memoria, la dirección de memoria a la que se apunta, una señal que indica si debe escribirse y la dirección de la siguiente instrucción.

La interfaz es:

```
IN  inM[16], instruction[16], reset
OUT outM[16], writeM, addressM[15], pc[15]
```

1. TEORÍA NECESARIA PARA CONSTRUIR CPU

La instrucción Hack tiene 16 bits. El bit instruction[15] permite distinguir los dos tipos de instrucciones existentes.

instruction[15]	Tipo	Función principal
0	A-instruction	Cargar un valor de 15 bits en el registro A
1	C-instruction	Realizar una operación del ALU y, opcionalmente, guardar el resultado o saltar

Cuando instruction[15] es 0, los otros 15 bits representan directamente el valor que se cargará en A. Cuando instruction[15] es 1, los demás bits se interpretan como campos de control de la instrucción C.

2. ESTRUCTURA DE UNA C-INSTRUCTION

La C-instruction puede dividirse conceptualmente así:

111 a c1 c2 c3 c4 c5 c6 d1 d2 d3 j1 j2 j3

Bits	Campo	Función
15..13	111	Identifican una C-instruction
12	a	Selecciona A o M como entrada y del ALU
11..6	c1..c6	Controlan la operación del ALU
5	d1	Indica destino A
4	d2	Indica destino D
3	d3	Indica destino M

Bits	Campo	Función
2..0	j1..j3	Determinan las condiciones de salto

El desarrollo de CPU consiste, por tanto, en convertir estos bits de la instrucción en señales que controlen los diferentes componentes.

3. REGISTRO A Y CARGA DE A

El registro A tiene una función doble. En una A-instruction almacena el valor contenido en `instruction[14..0]`. En una C-instruction puede almacenar el resultado del ALU cuando la instrucción especifica `dest=A`.

Para determinar cuándo debe cargarse A se consideran dos situaciones: si la instrucción es una A-instruction o si es una C-instruction cuyo bit `d1` está activado.

```
Not(in=instruction[15], out=esA);
And(a=instruction[15], b=instruction[5], out=esCdestA);
Or(a=esA, b=esCdestA, out=loadA);
```

La variable `esA` representa que `instruction[15]=0`. `esCdestA` representa que se trata de una C-instruction y que `d1=1`. La unión de ambas condiciones produce `loadA`.

4. MUX PARA LA ENTRADA DEL REGISTRO A

Ahora hay que decidir qué dato entra al registro A. En una A-instruction debe entrar `instruction`. En una C-instruction con `dest=A` debe entrar el resultado del ALU.

```
Mux16(
    a=instruction,
    b=outM,
    sel=instruction[15],
    out=mux1
);
```

La selección sigue la regla del Mux: `sel=0` deja pasar `a` y `sel=1` deja pasar `b`. Por ello una A-instruction selecciona `instruction` y una C-instruction selecciona el resultado del ALU. Cuando `loadA` es 0, el valor seleccionado por este Mux no modifica A, por lo que su contenido no cambia.

5. REGISTRO A Y `addressM`

El registro A también determina la dirección de memoria que está siendo utilizada por la CPU. Por eso su salida se conecta a `addressM`.

```
ARegister(
    in=mux1,
    load=loadA,
    out=A,
    out[0]=addressM[0],
    ...
    out[14]=addressM[14]
);
```

Esto significa que `addressM` no es una dirección calculada independientemente: corresponde al contenido actual del registro A.

6. REGISTRO D

El registro D almacena resultados intermedios y datos utilizados por el ALU. Solamente se carga cuando la instrucción es una C-instruction y d2=1. Como d2 corresponde a instruction[4], la señal de carga se obtiene mediante:

```
And(a=instruction[15], b=instruction[4], out=loadD);
```

El dato que se almacena en D es el resultado del ALU.

```
DRegister(in=outM, load=loadD, out=D);
```

7. SELECCIÓN ENTRE A Y M PARA EL ALU

Una de las partes fundamentales de la CPU es determinar qué debe recibir el ALU como entrada y. El campo a de la C-instruction se encuentra en instruction[12]. Si a=0, el ALU utiliza A. Si a=1, utiliza M, es decir, el contenido de memoria ubicado en la dirección A.

```
Mux16(
    a=A,
    b=inM,
    sel=instruction[12],
    out=mux2
);
```

Esta conexión es especialmente importante porque A e inM no representan lo mismo. A es una dirección/valor almacenado en el registro A, mientras que inM es el contenido de la memoria en esa dirección.

8. ALU

Una vez seleccionada la entrada y y teniendo D como entrada x, el ALU recibe los seis bits de control c1..c6.

```
ALU(
    x=D,
    y=mux2,
    zx=instruction[11],
    nx=instruction[10],
    zy=instruction[9],
    ny=instruction[8],
    f=instruction[7],
    no=instruction[6],
    out=outM,
    zr=zr,
    ng=ng
);
```

La salida outM representa el resultado de la operación. Las salidas zr y ng son necesarias posteriormente para determinar si debe realizarse un salto.

9. WRITE M

El bit d3, ubicado en instruction[3], indica si el resultado del ALU debe escribirse en memoria. Como únicamente una C-instruction puede realizar una escritura, se combinan instruction[15] e instruction[3].

```
And(a=instruction[15], b=instruction[3], out=writeM);
```

Cuando writeM=1, el valor outM debe escribirse en la dirección addressM. La CPU no escribe directamente en Memory: entrega outM, writeM y addressM al chip superior Computer.

10. LÓGICA DE SALTOS

Los últimos tres bits de una C-instruction determinan si el programa debe saltar a la dirección almacenada en A. Para decidirlo se utilizan las salidas zr y ng del ALU.

Bit	Condición
instruction[2]	JLT: salto si el resultado es negativo
instruction[1]	JEQ: salto si el resultado es cero
instruction[0]	JGT: salto si el resultado es positivo

Un resultado positivo se identifica cuando no es negativo y tampoco es cero. Por eso puede construirse con NOT, OR y AND.

```
And(a=instruction[2], b=ng, out=saltoNeg);
And(a=instruction[1], b=zr, out=saltoCero);

Or(a=ng, b=zr, out=ceroONeg);
Not(in=ceroONeg, out=esPositivo);

And(a=instruction[0], b=esPositivo, out=saltoPos);

Or(a=saltoNeg, b=saltoCero, out=saltoParcial);
Or(a=saltoParcial, b=saltoPos, out=saltoTotal);

And(a=instruction[15], b=saltoTotal, out=pcLoad);
```

La última condición incluye instruction[15] para garantizar que la lógica de salto solamente afecte a C-instructions.

11. FUNCIONAMIENTO DEL PC

El Program Counter (PC) contiene la dirección de la instrucción que debe ejecutarse. Para comprender su funcionamiento hay que tener clara la prioridad de sus señales: reset tiene prioridad, después load y finalmente inc.

Condición	Acción del PC
reset=1	PC = 0
reset=0 y load=1	PC = in
reset=0, load=0, inc=1	PC = PC + 1

En CPU, cuando se produce un salto, el PC debe cargar el valor de A. Cuando no se produce, debe incrementarse para pasar a la siguiente instrucción.

```
PC(
    in=A,
    load=pcLoad,
    inc=true,
    reset=reset,
    out=pc
);
```

De esta manera, A cumple también la función de almacenar el destino de un salto.

12. FLUJO COMPLETO DE UNA C-INSTRUCTION

El proceso puede resumirse de la siguiente manera. Primero se identifica que `instruction[15]=1`. Después se decodifican los bits de destino, los bits de control del ALU y los bits de salto. `instruction[12]` decide si será A o M. D proporciona x. El ALU calcula `outM`. Dependiendo de `d1`, `d2` y `d3`, el resultado puede cargarse en A, D o memoria. Finalmente, las salidas `zr` y `ng`, junto con `j1..j3`, determinan si PC debe cargar A o simplemente incrementarse.

13. EJEMPLO CONCEPTUAL

Supongamos una instrucción cuya operación debe calcular `D+M` y cuyo destino sea M. En primer lugar `instruction[15]=1`, por lo que es una C-instrucción. Como `a=1`, el `Mux16` selecciona `inM` para la entrada y del ALU. Los bits `c` controlan el ALU para realizar `D+M`. Como `d3=1`, `writeM` se activa. Por tanto:

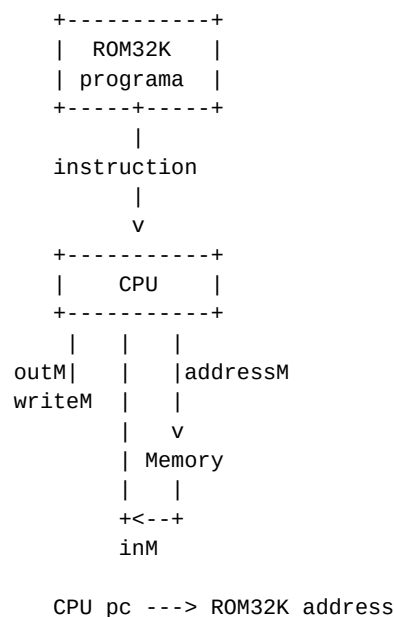
```
outM = D + M
writeM = 1
addressM = A
```

Computer utilizará estas tres señales para realizar la escritura en `Memory[A]`.

14. Computer.hdl

Después de construir CPU, el siguiente paso consiste en conectar los componentes principales del computador Hack. Computer no realiza nuevamente la lógica de la CPU ni la lógica de Memory; su función es interconectar CPU, ROM32K y Memory para formar el computador completo.

La estructura conceptual es:



15. ROM32K Y CPU

ROM32K contiene las instrucciones del programa. La CPU genera `pc`, que funciona como dirección de la instrucción que debe obtenerse. Por tanto:

```
ROM32K(
    address=pc,
    out=instruction
);
```

La salida `instruction` de ROM32K se conecta directamente a la entrada `instruction` de CPU. Esto crea el ciclo fundamental de ejecución: el PC determina qué instrucción se lee, la CPU ejecuta esa instrucción y calcula el siguiente PC.

16. MEMORY Y CPU

CPU también necesita comunicarse con Memory. Cuando una instrucción utiliza M, la CPU necesita conocer el contenido almacenado en la dirección A. Ese dato llega mediante inM.

```
Memory(  
    in=outM,  
    load=writeM,  
    address=addressM,  
    out=inM  
);
```

La conexión tiene una interpretación directa: outM es el dato que CPU quiere escribir, writeM indica si efectivamente debe escribir y addressM indica en qué dirección. Por otro lado, out de Memory se convierte en inM para que la CPU pueda leer memoria.

17. CONEXIÓN COMPLETA DE COMPUTER

La construcción de Computer consiste entonces en conectar los tres componentes de acuerdo con el flujo de información:

```
ROM32K(  
    address=pc,  
    out=instruction  
);  
  
Memory(  
    in=outM,  
    load=writeM,  
    address=addressM,  
    out=inM  
);  
  
CPU(  
    inM=inM,  
    instruction=instruction,  
    reset=reset,  
    outM=outM,  
    writeM=writeM,  
    addressM=addressM,  
    pc=pc  
);
```

Las señales internas inM, outM, writeM, addressM, instruction y pc sirven como puentes entre los chips.

18. CICLO COMPLETO DEL COMPUTADOR

Con todos los componentes conectados, el funcionamiento puede seguirse paso a paso.

Paso	Proceso
1	PC contiene la dirección de la instrucción actual.
2	ROM32K utiliza PC como address y entrega instruction.
3	CPU recibe instruction.
4	CPU puede leer Memory mediante inM si la instrucción utiliza M.
5	CPU ejecuta la operación mediante A, D y ALU.
6	Si dest=M, CPU produce outM y writeM=1.

Paso	Proceso
7	Memory utiliza addressM para determinar dónde escribir.
8	CPU determina el siguiente PC mediante inc o salto.
9	El proceso vuelve a comenzar con la nueva instrucción.

Este proceso forma el ciclo básico fetch-decode-execute: obtener la instrucción, interpretarla y ejecutarla.

19. RELACIÓN ENTRE MEMORY, CPU Y COMPUTER

La construcción de los tres chips puede verse como una progresión. Memory resuelve el acceso a diferentes dispositivos utilizando una dirección global. CPU utiliza los bits de instruction para generar señales de control y ejecutar operaciones. Finalmente, Computer conecta ambos junto con ROM32K.

Chip	Problema que resuelve
Memory.hdl	¿A qué dispositivo de memoria se accede y qué dato debe entrar o salir?
CPU.hdl	¿Qué operación indica la instrucción y qué deben hacer A, D, ALU y PC?
Computer.hdl	¿Cómo se conectan CPU, ROM y Memory para formar el computador completo?

20. DIFICULTADES Y PUNTOS IMPORTANTES

Uno de los puntos que requiere mayor cuidado en CPU es no confundir los diferentes bits de instruction. instruction[12] controla la selección A/M del ALU, instruction[5] controla el destino A, instruction[4] controla el destino D, instruction[3] controla el destino M y instruction[2..0] controlan los saltos.

Otro punto importante es la diferencia entre A e inM. A es el contenido del registro A y además representa la dirección que se envía a Memory. inM, en cambio, es el contenido de Memory en esa dirección.

También es importante no duplicar conexiones de una misma salida del ALU. El resultado debe tener una única salida interna, por ejemplo outM, y desde ella alimentar tanto el destino D como el destino A cuando corresponda.

En el PC también debe respetarse la prioridad de las señales. reset no debe competir con load; reset debe tener prioridad. Cuando no hay salto, el PC se incrementa normalmente.

21. CONCLUSIÓN

Para construir CPU.hdl primero se identificaron las partes de una C-instruction y se tradujeron sus bits a señales de control. Después se conectaron los registros A y D, el Mux que selecciona A o M, el ALU, la señal writeM y finalmente la lógica de saltos y el PC.

Una vez terminada CPU, Computer.hdl se obtiene conectando CPU con ROM32K y Memory. ROM32K entrega la instrucción determinada por pc; CPU interpreta y ejecuta esa instrucción; Memory proporciona los datos solicitados o recibe los datos que CPU desea escribir.

De esta manera, CPU y Computer siguen la misma idea utilizada anteriormente en Memory: primero se analiza qué representa cada señal, después se determina qué componente debe procesarla y finalmente se conectan los componentes mediante las señales de control correspondientes. El resultado es un

sistema en el que cada chip tiene una responsabilidad específica, pero todos trabajan conjuntamente para ejecutar un programa Hack.